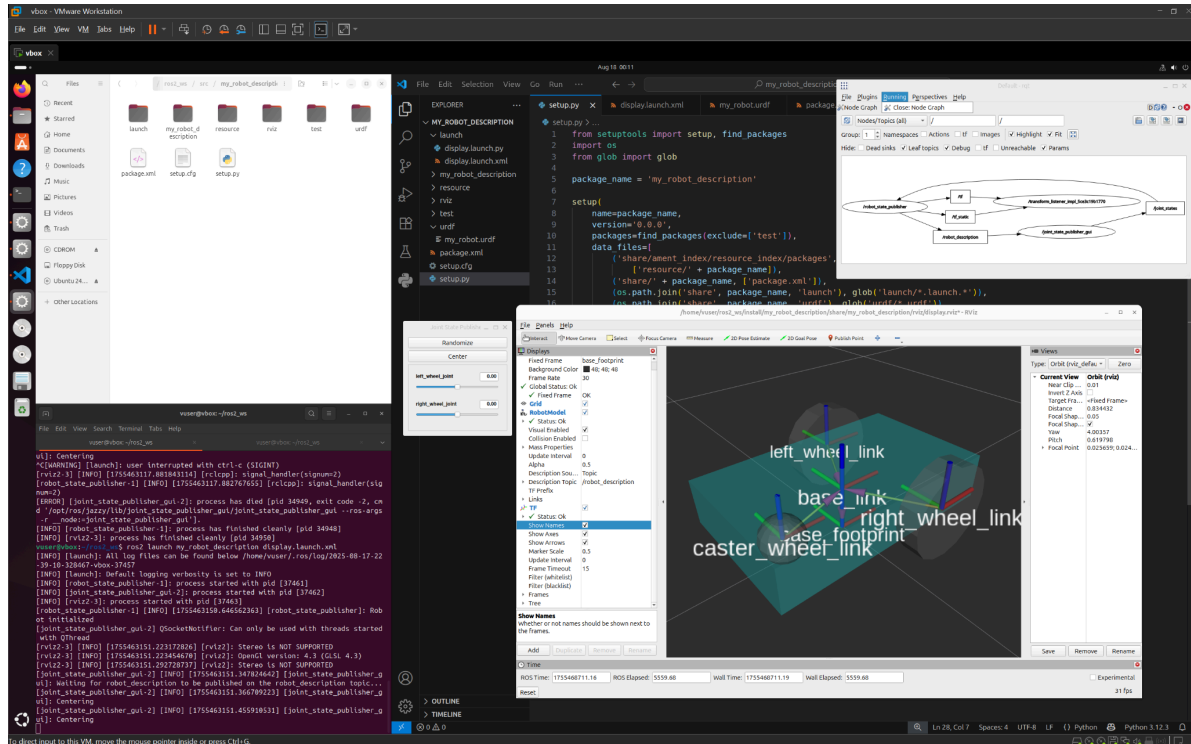


Assignment 1

This assignment aims to provide hands-on experience with the Robot Operating System (ROS) and the Unified Robot Description Format (URDF). Your group will model a simple yet common mobile robot platform: a differential-drive robot with a front caster wheel for stability (see Modern Robotics chapter 13.3.1.2 - The Differential-Drive Robot). You will define its physical structure, visualize it in RViz, and implement basic wheel movement.



Task Breakdown

Task 1: Create the URDF File

Your first task is to write a complete URDF file (**.urdf**) that describes the robot's structure. Some of you might be tempted to make a 5cm long robot with a mass four times that of the sun, but please try to keep it physically plausible. When we simulate this platform later, it needs to be somewhat physically accurate.

[URDF — ROS 2 Documentation: Jazzy documentation](#)

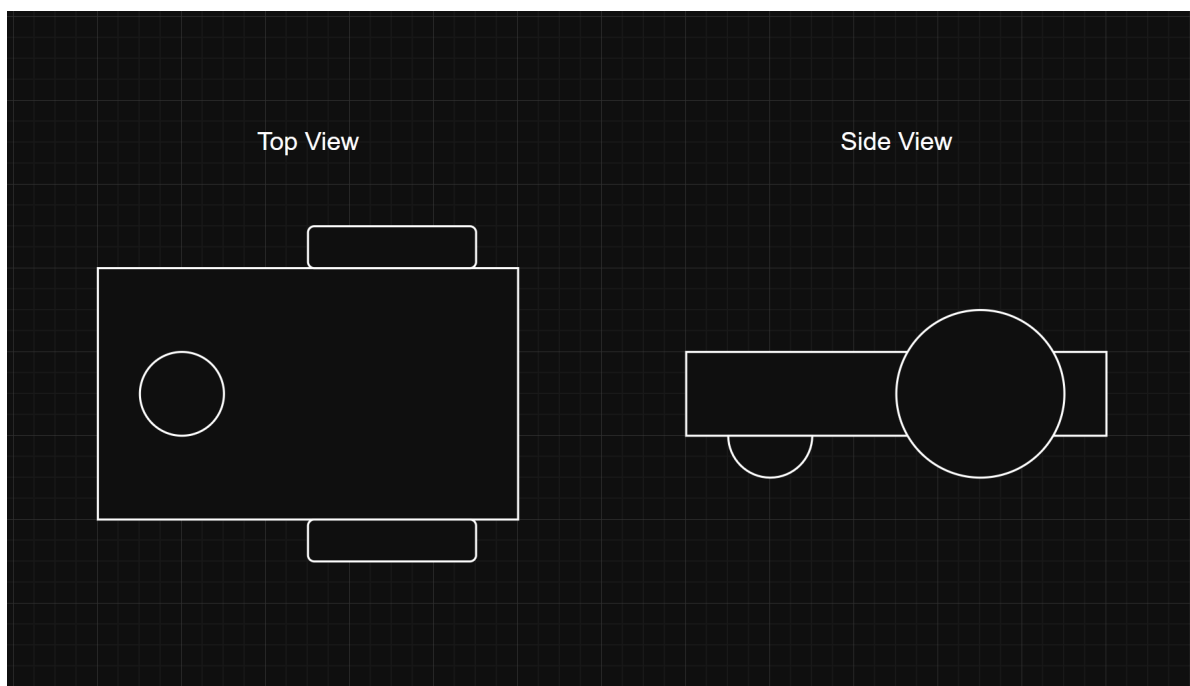
[urdf - ROS Wiki](#)

[urdf/XML/model - ROS Wiki](#)

[urdf/XML/joint - ROS Wiki](#)

- Base Footprint (**base_footprint**): The very root of our TF tree.
 - It will not have any visual, collision, or inertial tags.

- It will only serve as a virtual joint, allowing us to define a distance to the **base_link** with a fixed joint (**base_joint**).
- Base Link (**base_link**): This will be the main chassis of your robot
 - Define its geometry (e.g., a box or cylinder).
 - Define its visual properties (material).
- Drive Wheels (**left_wheel** & **right_wheel**):
 - Create two identical links for the wheels. Define their cylindrical geometry and visual properties.
 - Connect each wheel to the **base_link** using a **continuous** joint. This allows them to rotate indefinitely.
 - Position the joints correctly relative to the **base_link**. The axis of rotation for both wheels should be the same.
- Caster Wheel (**caster_wheel**):
 - Create a link for the caster wheel. A sphere is a good choice for its geometry.
 - Connect the caster wheel to the **base_link** using a **fixed** joint. For this assignment, we will assume the caster wheel is a sphere that can roll in any direction.
 - Position the joint correctly at the front of the robot.



Task 2: Read the wiki

You'll need to create a package called `my_robot_description` (use `ament_python` to add Python as a dependency), build it, source it, and then put the URDF in a subfolder called `urdf`, and the launch file in a subfolder called `launch`. Please take some time to read the documentation on creating a package, writing a URDF, and creating a launch file (in xml). Google is your friend; there are many pitfalls in ROS, and you'll have to look up some stuff.

[Creating a workspace — ROS 2 Documentation: Jazzy documentation](#)

[Creating a package — ROS 2 Documentation: Jazzy documentation](#)

[Using colcon to build packages — ROS 2 Documentation: Jazzy documentation](#)

Task 3: Visualization and TF Tree

Next, you will visualize your robot model to verify its correctness.

[Creating a launch file — ROS 2 Documentation: Jazzy documentation](#)

- Launch File: Create a ROS launch file (`.launch.xml`) that accomplishes the following. I recommend making an XML launch file instead of a Python one, but that's just personal preference.
 - Start the `robot_state_publisher` and `joint_state_publisher` nodes.
 - The `robot_state_publisher` takes your urdf and publishes it on the graph so that other applications, such as RViz, know where to find it.
 - The `joint_state_publisher` provides a GUI to manually control the joints.
 - Start RViz as a node with a configuration file that displays your robot model with transforms (TFs).
- TF Tree:
 - With your launch file running, use the `tf2_tools` to generate a PDF of your robot's TF tree (or figure out how to do it with `rqt`).
 - `ros2 run tf2_tools view_frames`
 - `evince frames.pdf`
- RViz Screenshot:
 - In RViz, display the robot model.
 - Enable the TF display and set the "Show Names" and "Show Axes" properties to true.

- You can also reduce the size of the axes and the alpha of the robot model to get a better representation, similar to the one in the top screenshot.
- Capture a screenshot showing the robot model with all its transform axes clearly visible.

Task 4: Adding Wheel Motion

Finally, you will write a simple Python script to publish joint states and make the wheels spin.

- Create a Publisher Node: Write a Python script that creates a ROS node.
- Publish Joint States: The node should continuously publish `sensor_msgs/JointState` messages to the `/joint_states` topic.
- Simulate Motion: In your script, increment the position of the `left_wheel` and `right_wheel` joints over time to make them rotate. The caster wheel joint is fixed and does not require updating.
- Update Launch File: Modify your launch file to run this new node instead of the `joint_state_publisher` GUI.

Delivery

I strongly recommend using Overleaf for your report.

You can have a look at this for some layout inspiration and how to add code snippets in your report:

<https://www.overleaf.com/read/gstrwxnkwnhy#0160ba>

I'm looking for a well-structured and documented URDF, figures of the TF tree, a robot model in RViz, and the full node tree when everything's running. Don't worry if everything isn't working - I'm looking for effort and a willingness to learn; we have plenty more to cover before the semester is over. I'm not looking for 15,000 words; I want to see well-structured reports that quickly convey the needed information in a professional and elegant manner. Try channeling your favorite science communicator; I should be convinced that you've understood what you've done.

Good luck! :)